</> CodEval

Créer vos devoirs et examens

Guide enseignant, pas à pas : saisir dans CodEval le devoir d'Initiation algorithmique, le devoir et l'examen de C avancé.



DEVOIR COMMUN · SRIT1

Initiation algorithmique

- Simulation d'un algorithme
- 3 algorithmes à écrire
- 2 h · 20 points

DEVOIR SURVEILLÉ · M1 MBDS

C avancé

- Questions de cours
- Approche modulaire rédigée
- 2 h · 20 points

EXAMEN 2e SESSION · M1

C avancé

- Projet C en 5 fichiers
- Commandes de compilation
- QCM de 5 questions

Comment lire ce guide

Chaque exercice est présenté en deux temps : **sur le sujet papier** (l'extrait du PDF), puis **dans CodEval** (la capture annotée). Les **pastilles rouges numérotées** des captures renvoient aux étapes écrites sous chaque image. Un type d'exercice n'est détaillé qu'une fois : pour les exercices du même type, seul ce qui change est indiqué.

1. Vue d'ensemble : 4 étapes, 5 types d'exercices
2. Avant de commencer : matière et classe
3. Créer l'épreuve : l'étape « Paramètres »
4. Ajouter un exercice : choisir le type

A. Devoir d'Initiation algorithmique

B. Devoir surveillé de C avancé

C. Examen 2e session de C avancé

D. Après l'épreuve : correction et copies

E. Aide-mémoire

1. Vue d'ensemble

Toute épreuve se crée en quatre étapes, toujours dans le même ordre.

1. **Paramètres** — Titre, type, classe, matière, durée, barème, consignes
2. **Exercices** — Le type de chaque exercice, son énoncé, ses questions ou son tableau
3. **Correction** — Pour les exercices pratiques : déclarations attendues et jeux de tests
4. **Points & publication** — La valeur de chaque exercice, puis publier

Quel type pour quel exercice ?

Exercice du sujet	Type dans CodEval	Correction
Algo, ex. 1 : simulation de *CalculDeMoyenne*	**Simulation** nouveau	Automatique, case par case
Algo, ex. 2, 3 et 4 : écrire un algorithme	**Algorithmique**	Automatique, jeux de tests
C, devoir, ex. 1 : questions de cours **C, devoir**, ex. 2 : approche modulaire (gestion de stock) **C, examen**, ex. 2.2 : commandes de compilation	**Question-réponse**	Mots-clés, formulations acceptées, ou correction manuelle
C, examen, ex. 1 et 2.1 : Etudiant.h, Note.h, Fonctions.h, Fonctions.c, main.c	**Code (plusieurs fichiers)**	Automatique : déclarations + programme lancé avec un argument
C, examen, ex. 3 : cinq questions à choix unique	**QCM**	Automatique

Deux nouveautés ajoutées pour ces sujets

Le type Simulation : un vrai tableau de simulation, corrigé case par case (avant, il fallait transformer l'exercice en questions).

La comparaison « Nombres affichés » : la correction d'un algorithme ne regarde plus que les nombres affichés à la fin. L'étudiant écrit ses messages comme il veut (« Entrez un nombre : », « Le plus grand est : »…) sans perdre de points.

2. Avant de commencer

La matière et la classe, une fois pour toutes.

Une épreuve est rattachée à une **matière**, et c'est la matière qui fixe le langage. Créez-les une fois pour toutes dans le menu **Mes matières** :

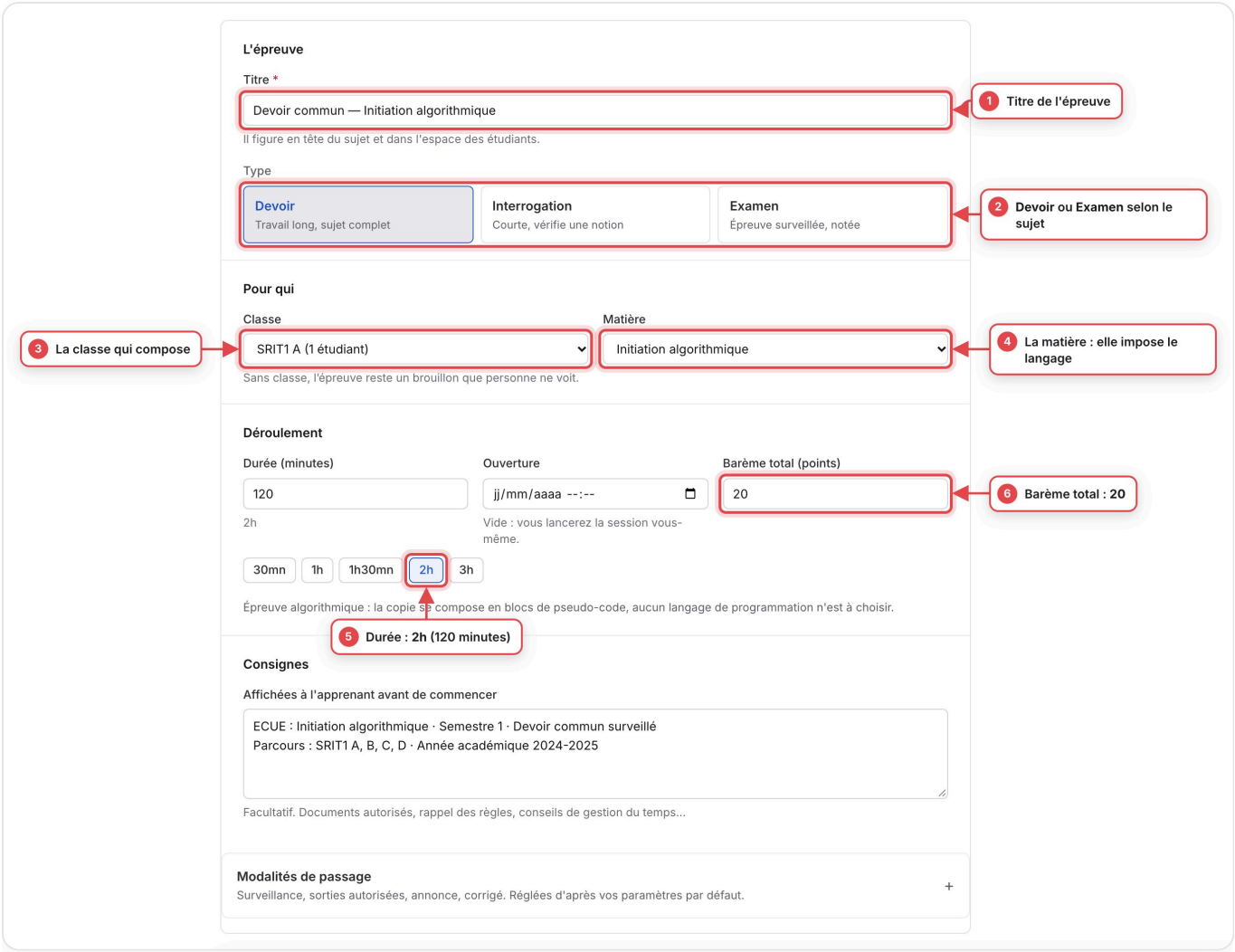


1. Saisissez le nom : *Initiation algorithmique*, puis *C avancé*.
2. Choisissez le langage : **Algorithmique** (pseudo-code en blocs) ou **C**.
3. Cliquez sur **Ajouter**.

Créez aussi vos **classes** (menu **Mes classes**, par exemple *SRIT1 A* et *M1 MBDS*). Les étudiants les rejoignent avec le code de la classe.

3. Créer l'épreuve

Menu **Créer une évaluation**, étape 1 « Paramètres ».



1. Le **titre**, repris en tête du sujet et dans l'espace des étudiants.
2. Le **type** : *Devoir* pour les deux devoirs, *Examen* pour la 2e session.
3. La **classe** qui compose. Sans classe, l'épreuve reste un brouillon que personne ne voit.
4. La **matière**, qui impose le langage.
5. La **durée** : bouton *2h* (les trois sujets durent 2 h).
6. Le **barème total** : 20 points.

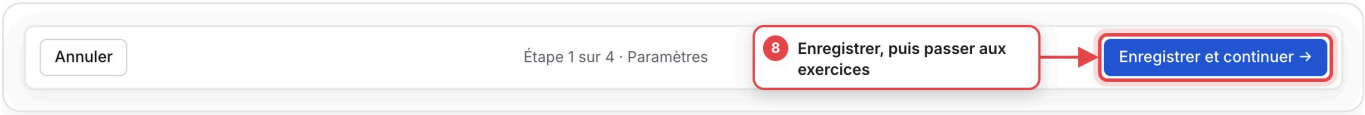
	Devoir d'algorithmique	Devoir de C	Examen de C
Titre	Devoir commun — Initiation algorithmique	Devoir surveillé — C avancé	Examen 2e session — C avancé
Type	Devoir	Devoir	Examen
Classe / matière	SRIT1 A / Initiation algorithmique	M1 MBDS / C avancé	M1 MBDS / C avancé
Durée · barème	2h · 20	2h · 20	2h · 20

Les consignes

Plus bas sur la même page, le champ **Consignes** reçoit l'en-tête du sujet et le bloc « Important à lire ». L'étudiant les lit avant de commencer.



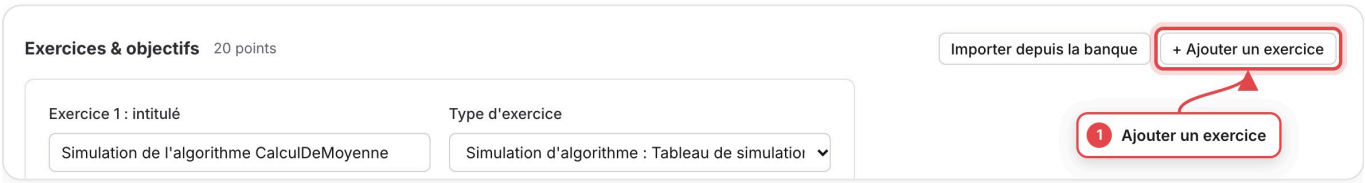
> **« 1 pt pour la propreté de la copie » (devoir de C)**
> Une copie tapée n'a pas de propreté à juger. Pour garder ce point, ajoutez-le à la main sur la copie de chaque étudiant avec **Modifier la note de cet exercice** (voir partie D). Chaque ajustement est enregistré avec son motif.



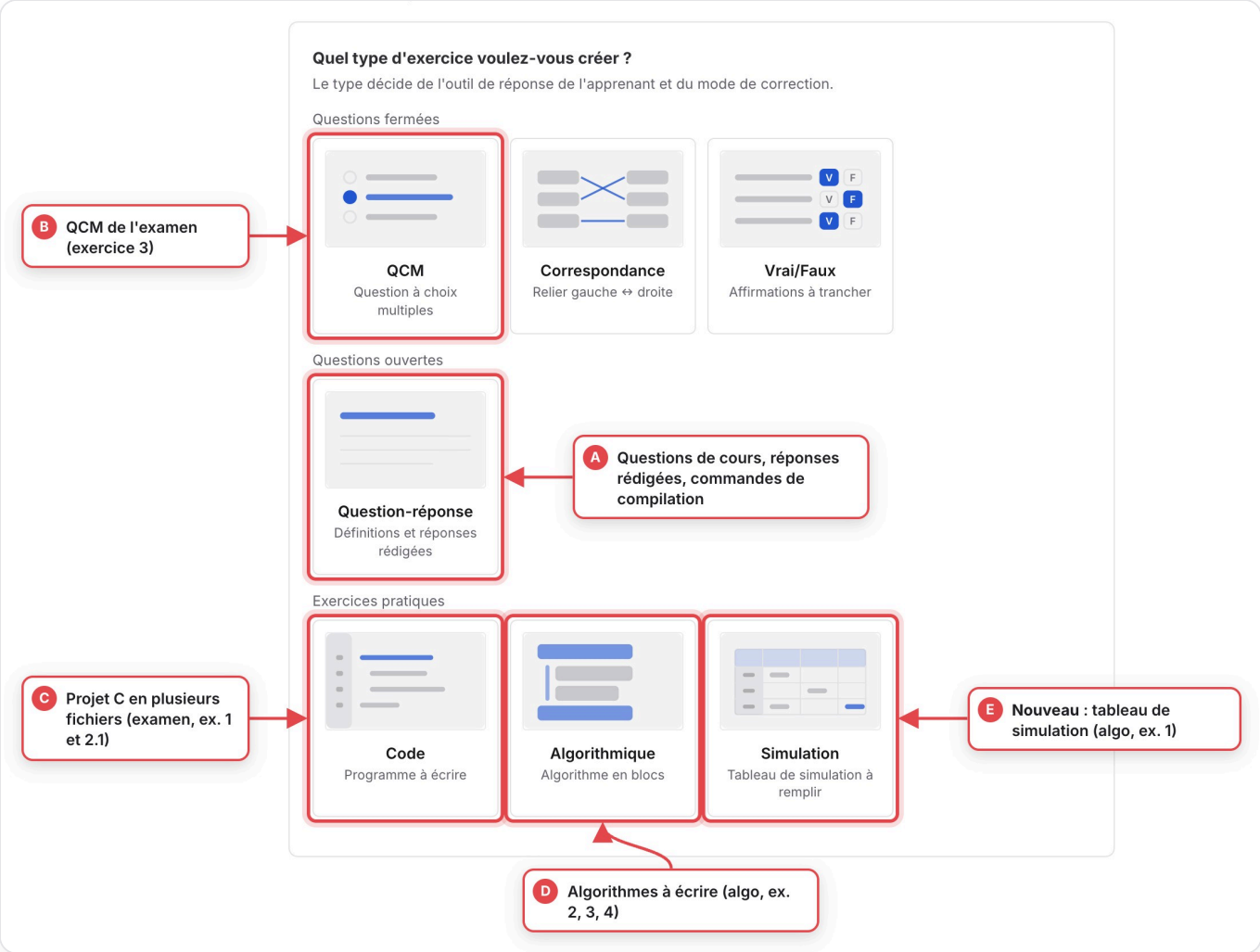
8 **Enregistrer et continuer** → enregistre les paramètres et ouvre l'étape « Exercices ».

4. Ajouter un exercice

Étape 2 « Exercices » : un exercice à la fois, en commençant par choisir son type.



Le bouton **+ Ajouter un exercice** affiche les types. Choisissez celui qui correspond à l'exercice du sujet :



A	**Question-réponse** : questions de cours, approche modulaire rédigée, commandes de compilation
B	**QCM** : exercice 3 de l'examen
C	**Code** : le programme modulaire de l'examen (exercices 1 et 2.1)
D	**Algorithmique** : exercices 2, 3 et 4 du devoir d'algorithmique
E	**Simulation** : exercice 1 du devoir d'algorithmique

> **L'aperçu du sujet**
>
> Pendant la saisie, la colonne de droite montre la feuille telle que l'étudiant la recevra. Le type d'un exercice reste modifiable avec la liste **Type d'exercice**, en haut de sa carte.

A. Devoir d'Initiation algorithmique

Devoir commun du 30/11/2024 · SRIT1 · 4 exercices · 20 points

SIMULATION Exercice 1 : simulation de CalculDeMoyenne (5 points)

SUR LE SUJET PAPIER

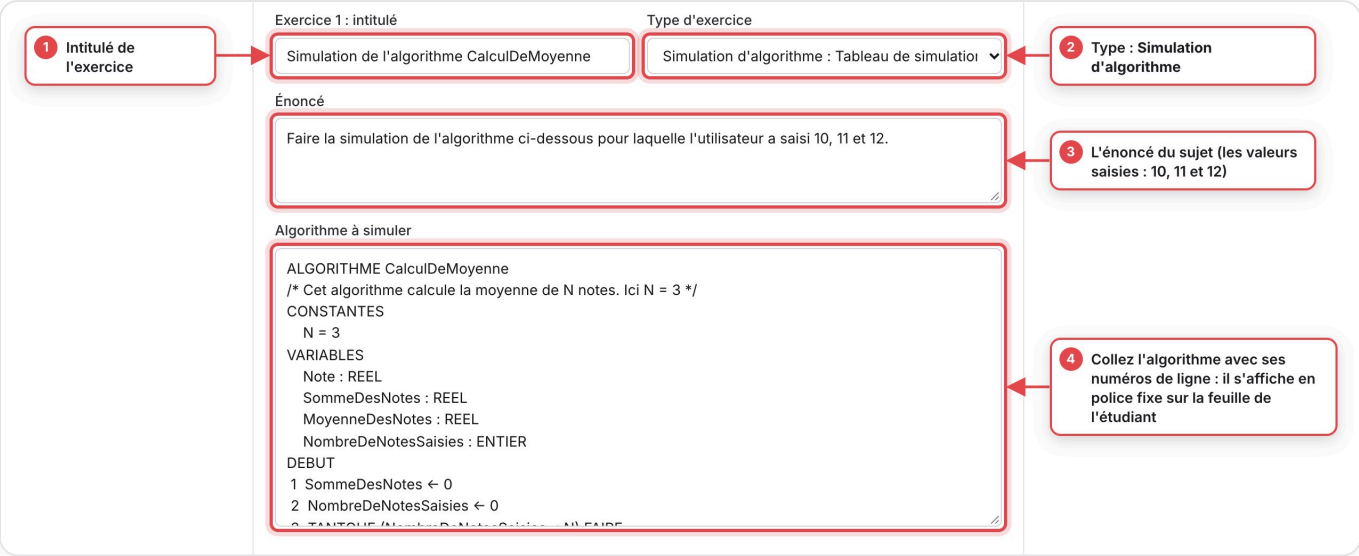


Exercice 1 : (5 points)

Faire la simulation de l'algorithme ci-dessous pour laquelle l'utilisateur a saisi **10, 11** et **12**.

```
ALGORITHME CalculDeMoyenne
|/*Cet algorithme calcule la moyenne de N notes. Ici N = 3*/
|       CONSTANTES
|       |       N = 3
|       VARIABLES
|       |       Note : REEL
|       |       SommeDeNotes : REEL
|       |       MoyenneDesNotes : REEL
|       |       NombreDeNotesSaisies : ENTIER
DEBUT
|1      SommeDesNotes ← 0
|2      NombreDeNotesSaisies ← 0
|3      TANTQUE (NombreDeNotesSaisies < N) FAIRE
|4      |       ECRIRE ("Note Numéro : ", NombreDeNotesSaisies +1)
|5      |       LIRE (Note)
|6      |       SommeDesNotes ← SommeDesNotes + Note
|7      |       NombreDeNotesSaisies ← NombreDeNotesSaisies + 1
|8      FINTANTQUE
|9      MoyenneDesNotes ← SommeDesNotes / N
|10     ECRIRE ("Moyenne des ", N, "note(s) : ", MoyenneDesNotes)
FIN
```

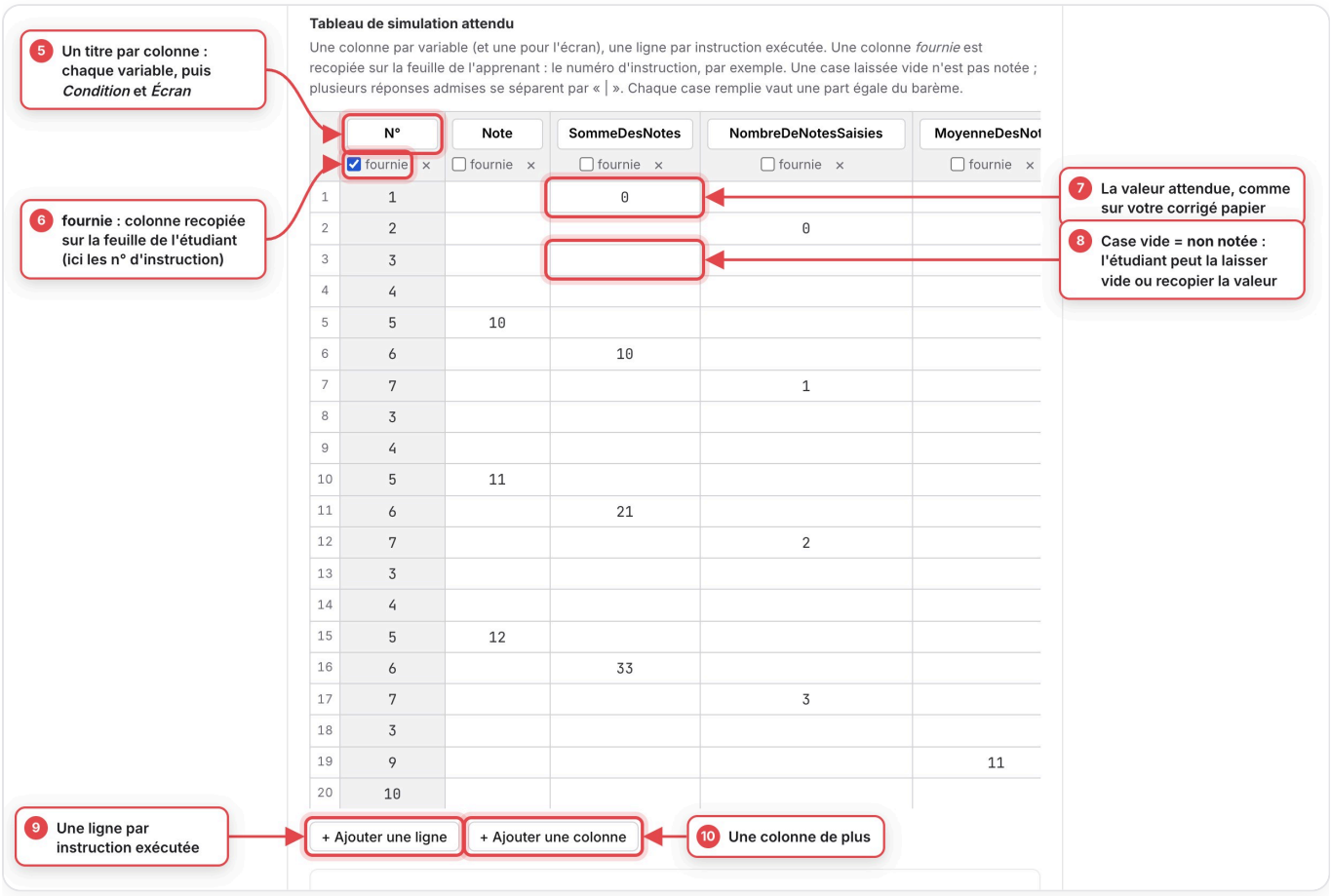
Dans CodEval : l'énoncé et l'algorithme



1. L'**intitulé** : « Simulation de l'algorithme CalculDeMoyenne ».
2. Le **type** : *Simulation d'algorithme*.
3. L'**énoncé** : « Faire la simulation de l'algorithme ci-dessous pour laquelle l'utilisateur a saisi 10, 11 et 12. »
4. L'**algorithme**, recopié avec ses numéros de ligne (1 à 10). L'étudiant le lit coloré, en police fixe.

Dans CodEval : le tableau de simulation attendu

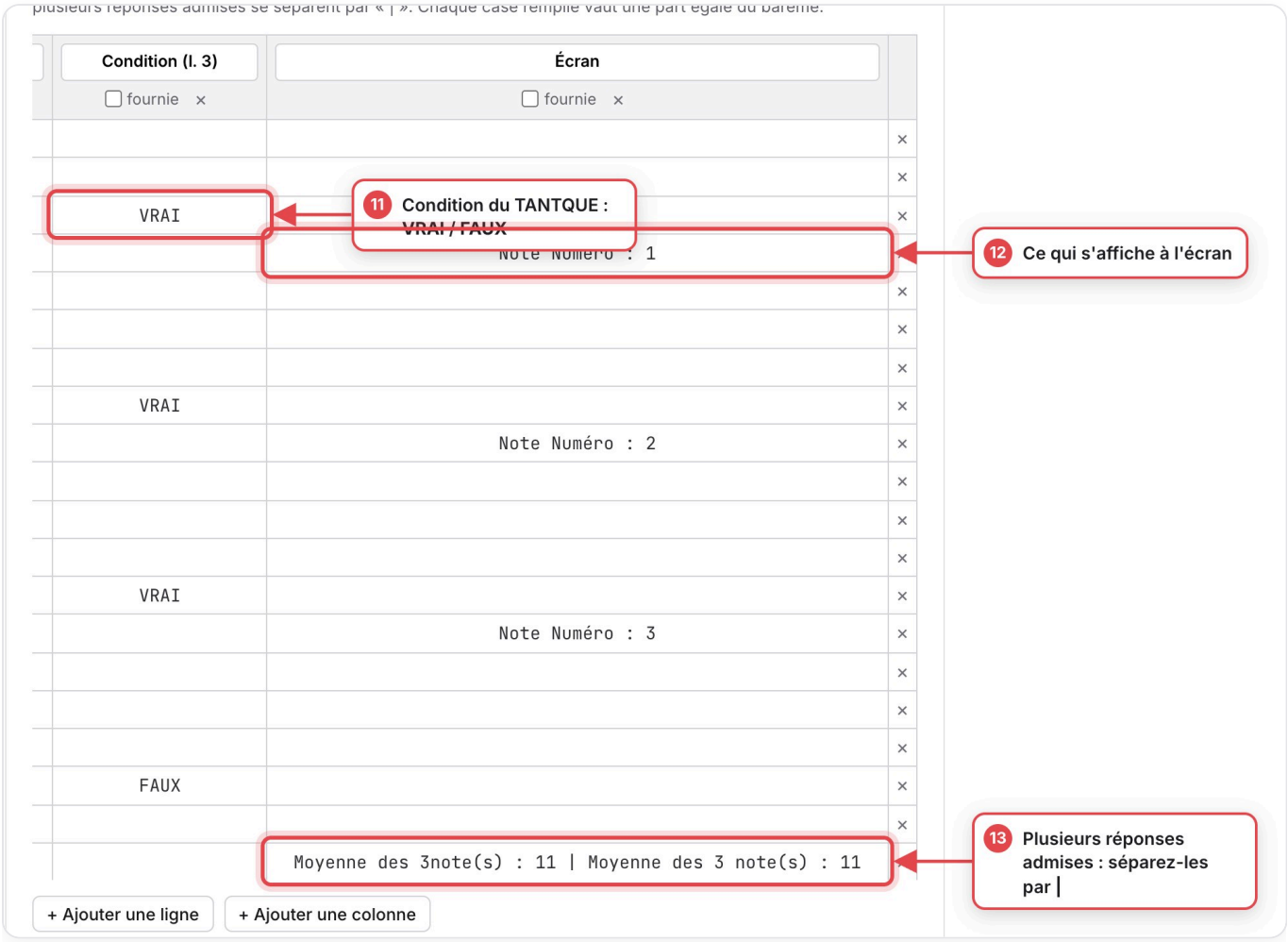
Remplissez le tableau comme votre corrigé papier : une colonne par variable, une colonne *Condition* pour le test du TANTQUE, une colonne *Écran*, et **une ligne par instruction exécutée**.



	N°	Note	SommeDesNotes	NombreDeNotesSaisies	MoyenneDesNot
	fournie	fournie	fournie	fournie	fournie
1	1		0		
2	2			0	
3	3				
4	4				
5	5	10			
6	6		10		
7	7			1	
8	3				
9	4				
10	5	11			
11	6		21		
12	7			2	
13	3				
14	4				
15	5	12			
16	6		33		
17	7			3	
18	3				
19	9				11
20	10				

5. Le **titre** de chaque colonne : N°, Note, SommeDesNotes, NombreDeNotesSaisies, MoyenneDesNotes, Condition (l. 3), Écran.
6. **fournie** : la colonne est recopiée sur la feuille de l'étudiant. Cochée ici pour les numéros d'instruction (1, 2, 3, 4, 5, 6, 7, 3, 4…). Décochez-la si l'étudiant doit retrouver lui-même l'ordre d'exécution : elle sera alors notée.
7. La **valeur attendue** dans la case.
8. Une **case vide n'est pas notée** : l'étudiant peut la laisser vide ou recopier la valeur précédente.
9. **+ Ajouter une ligne** : 20 lignes pour cet algorithme (3 tours de boucle).
10. **+ Ajouter une colonne** si une variable manque.

La partie droite du tableau (faites défiler horizontalement)



11 **Condition** : VRAI aux lignes 3 tant que la boucle continue, FAUX au dernier passage.

12 **Écran** : le texte affiché, « Note Numéro : 1 »…

13 **Plusieurs réponses admises** : séparez-les par `|`. Ici `Moyenne des 3note(s) : 11 | Moyenne des 3 note(s) : 11`.

> **Comment la simulation est corrigée**
>
> Chaque case remplie vaut une part égale du barème : 20 cases pour 5 points, soit 0,25 point par case. Les nombres se comparent par leur valeur (`10` = `10,0` = `10.00`). Le texte se compare sans tenir compte des majuscules, des accents ni de la ponctuation (`vrai` = `VRAI`). Le corrigé n'est jamais envoyé à l'étudiant avant la publication des notes.

ALGORITHMIQUE Exercices 2, 3 et 4 : écrire un algorithme

Les trois exercices se créent de la même façon. Exemple de l'exercice 4 :

SUR LE SUJET PAPIER



Exercice 4 : (6 points)

Ecrire un algorithme qui :

1- demande successivement plusieurs nombres à l'utilisateur, et qui lui dise ensuite quel était le plus grand parmi ces nombres ;
2- affiche ensuite la position à laquelle avait été saisie ce plus grand nombre

NB : On ne connaît pas d'avance combien l'utilisateur souhaite saisir de nombres. La saisie des nombres s'arrête lorsque l'utilisateur entre un zéro.

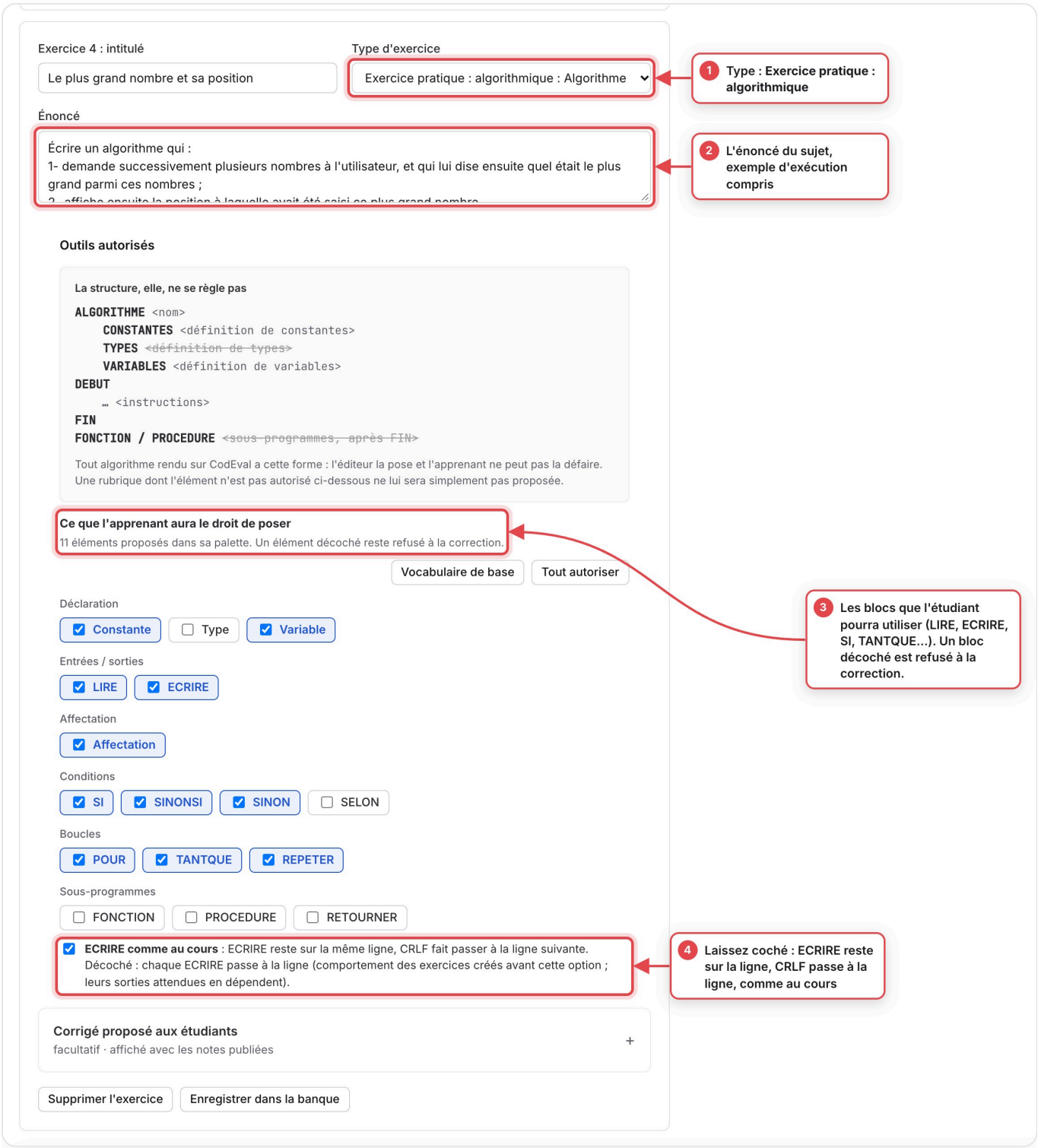
Exemple d'exécution :

Entrez le nombre numéro 1 : 12
Entrez le nombre numéro 2 : 5
Entrez le nombre numéro 3 : 15
Entrez le nombre numéro 4 : 10
Le plus grand de ces nombres est : 15
C'était le nombre numéro 3

Les exercices 2 et 3

Même création que l'exercice 4 ci-après : seuls l'énoncé et les tests changent (tableau page suivante).

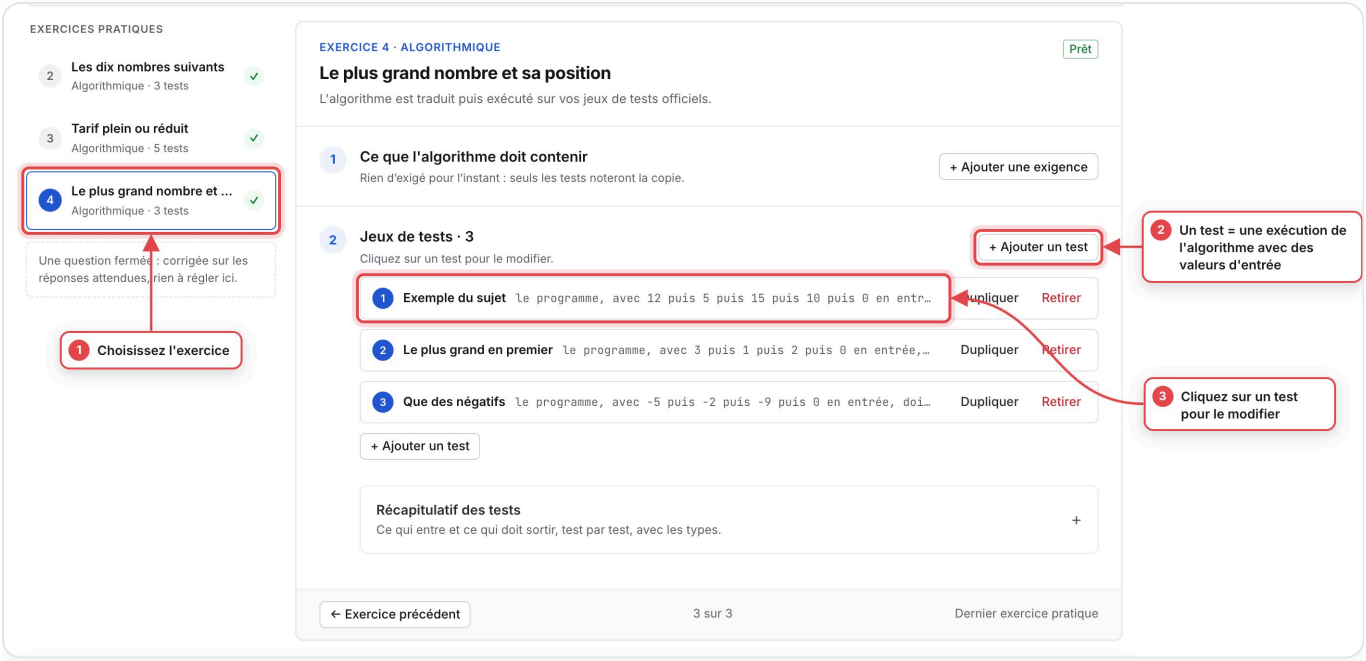
Dans CodEval : l'énoncé et les outils autorisés



1. Type **Exercice pratique : algorithmique**.
2. L'énoncé du sujet, exemple d'exécution compris.
3. Les **blocs autorisés** : LIRE, ECRIRE, SI, SINON, POUR, TANTQUE… Un bloc décoché n'apparaît pas dans la palette de l'étudiant et est refusé à la correction.
4. Laissez **ECRIRE comme au cours** coché.

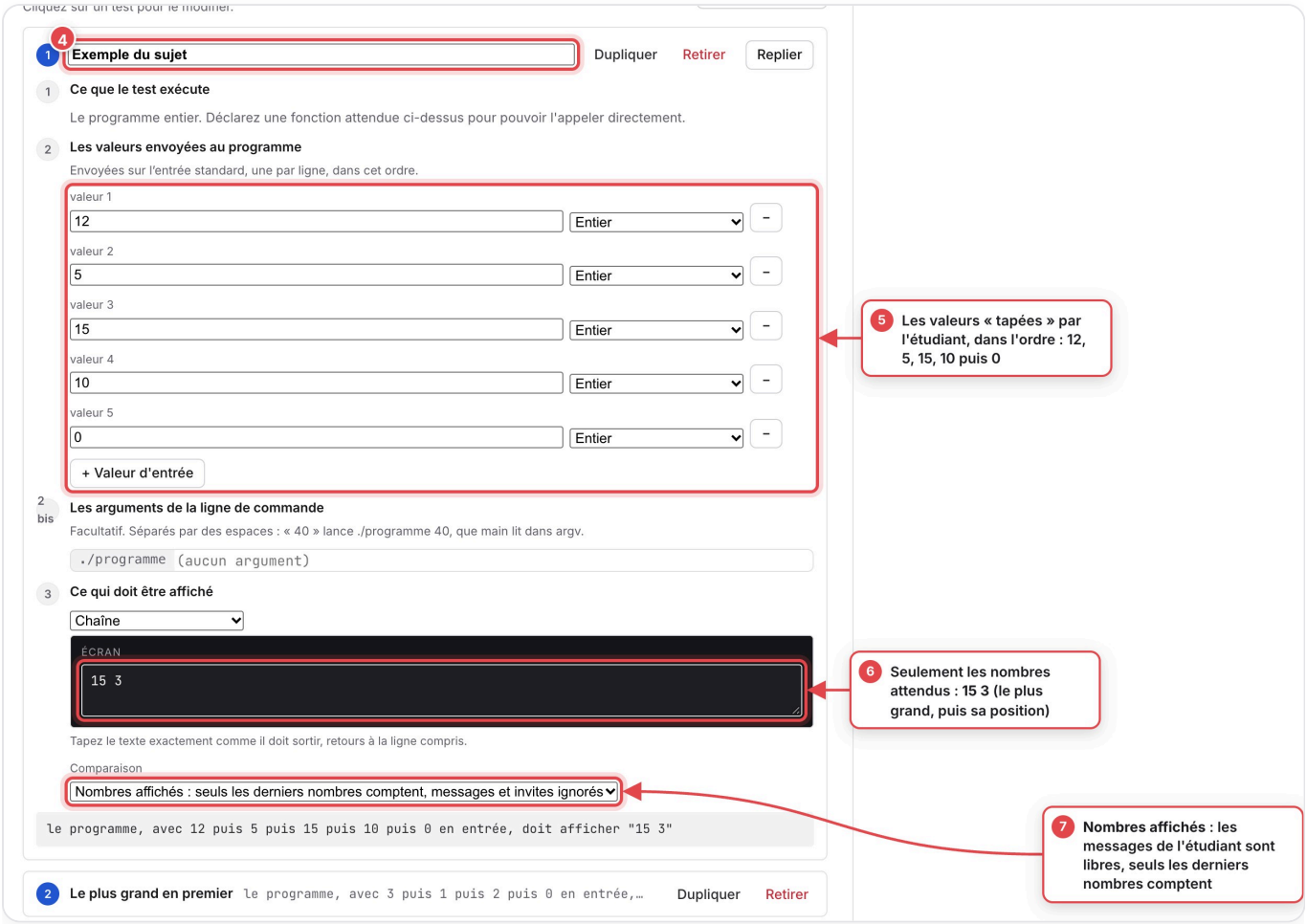
Étape 3 « Correction » : les jeux de tests

Après l'étape « Exercices », l'étape **Correction** liste les exercices pratiques. Un test exécute l'algorithme de l'étudiant avec des valeurs d'entrée, puis vérifie ce qu'il affiche.



1. Choisissez l'exercice dans la colonne de gauche.
2. **+ Ajouter un test** pour chaque cas à vérifier.
3. Cliquez sur un test pour l'ouvrir.

Un test ouvert



4. Le nom du test, libre : « Exemple du sujet ».
5. Les **valeurs envoyées**, une par ligne, dans l'ordre où l'étudiant les saisit : 12, 5, 15, 10, puis 0 pour arrêter.
6. Ce qui doit être affiché : seulement les **nombres attendus**, ici `15 3`.
7. La comparaison **Nombres affichés** : invites et messages ignorés, seuls les derniers nombres comptent.

Les tests des trois exercices

Exercice	Valeurs envoyées (un test par ligne)	Attendu · *Nombres affichés*
2. Les dix nombres suivants (5 pts)	`17`	`18 19 20 … 27`
	`-3`	`-2 -1 0 … 7`
	`0`	`1 2 3 … 10`
3. Tarif plein ou réduit (4 pts)	`mercredi` · `30` · `NON`	`4.5`
	`lundi` · `30` · `OUI`	`4.5`
	`samedi` · `12` · `NON`	`7`
	`mardi` · `70` · `NON`	`4.5`
	`dimanche` · `30` · `NON`	`7`
4. Le plus grand et sa position (6 pts)	`12 · 5 · 15 · 10 · 0`	`15 3`
	`3 · 1 · 2 · 0`	`3 1`
	`-5 · -2 · -9 · 0`	`-2 2`

EXERCICE 3 SUR LE SUJET

Exercice 3 : (4 points)

Il y a deux tarifs : plein (7 FCFA) et réduit (4,50 FCFA). Le tarif réduit s'applique :

- à tout le monde le mercredi,
- aux étudiants quel que soit leur âge et quel que soit le jour,
- aux moins de 18 ans et plus de 65 ans SAUF les samedi et dimanche.

Ecrire un **Algorithme** qui recueille les données nécessaires et annonce le prix à payer pour la place.

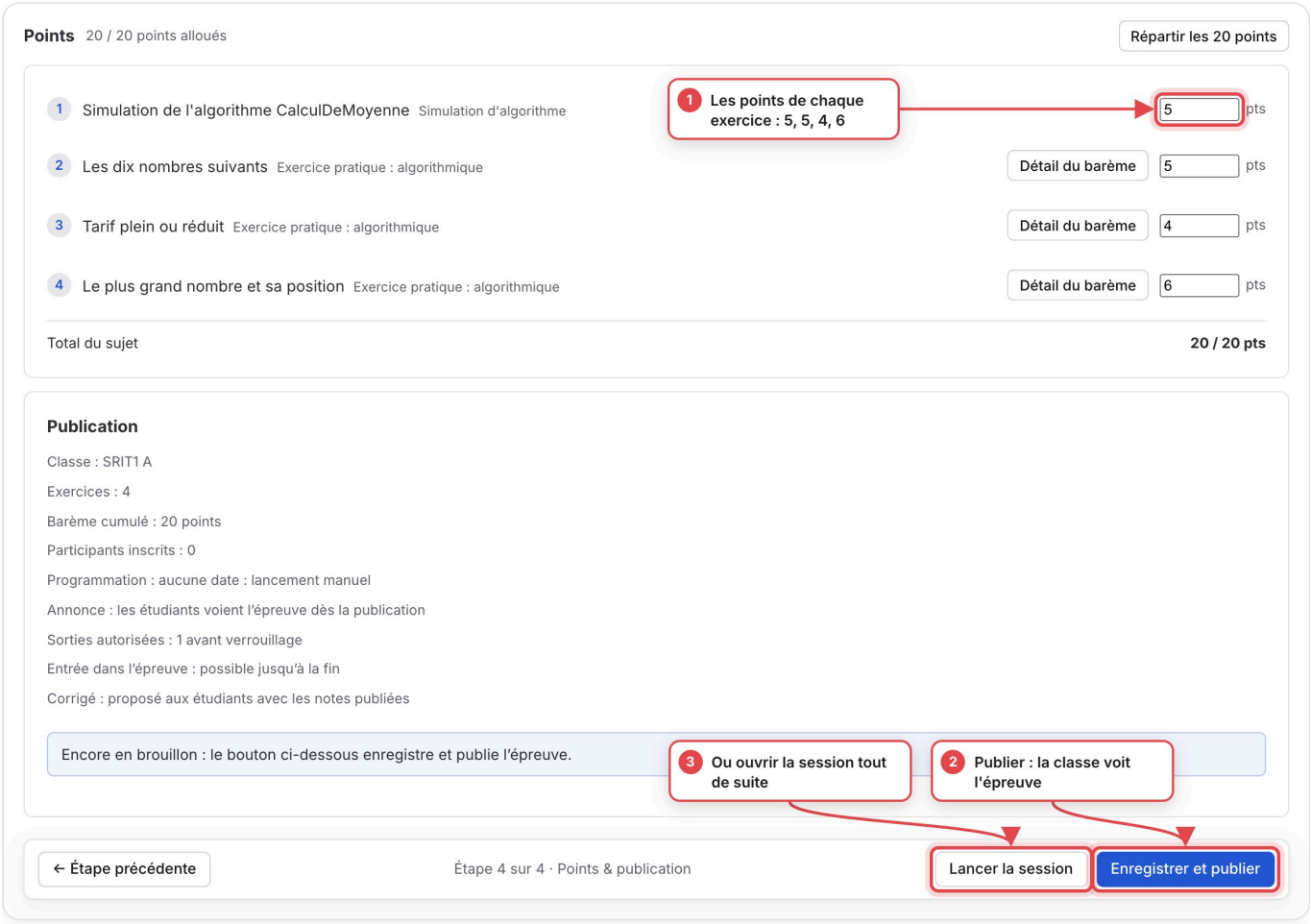
Exercice 3 : précisez l'ordre de saisie dans l'énoncé

Les tests envoient les valeurs dans un ordre fixe. Ajoutez une phrase à l'énoncé, par exemple « Données à saisir, dans cet ordre : le jour (lundi … dimanche), l'âge, puis OUI ou NON (étudiant ?) ». C'est la seule précision à ajouter au sujet : les messages restent libres.

Les autres comparaisons

Souple : texte exact, espaces de fin ignorés, pour un affichage imposé. *Numérique* : la sortie ne contient que des nombres. *Exacte* : caractère par caractère.

Étape 4 « Points & publication »

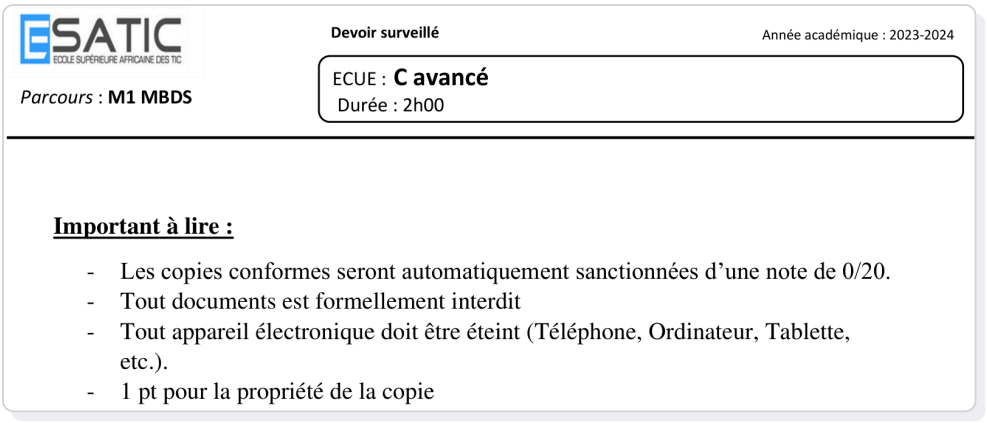


1. Les points de chaque exercice : **5, 5, 4, 6**. Le total doit faire 20.
2. **Enregistrer et publier** : la classe voit l'épreuve (avec une date d'ouverture, le bouton devient *Enregistrer et programmer*).
3. **Lancer la session** ouvre l'épreuve tout de suite.

B. Devoir surveillé de C avancé

Devoir du 27/02/2024 · M1 MBDS · 2 exercices · 20 points

SUR LE SUJET PAPIER



ESATIC
ECOLE SUPÉRIEURE AFRICAINE DES TIC

Parcours : **M1 MBDS**

Devoir surveillé

Année académique : 2023-2024

ECUE : **C avancé**
Durée : 2h00

Important à lire :

- Les copies conformes seront automatiquement sanctionnées d'une note de 0/20.
- Tout documents est formellement interdit
- Tout appareil électronique doit être éteint (Téléphone, Ordinateur, Tablette, etc.).
- 1 pt pour la propriété de la copie

Paramètres : type **Devoir**, classe *M1 MBDS*, matière *C avancé*, 2h, 20 points. Recopiez « Important à lire » dans les **Consignes**, comme au chapitre 3.

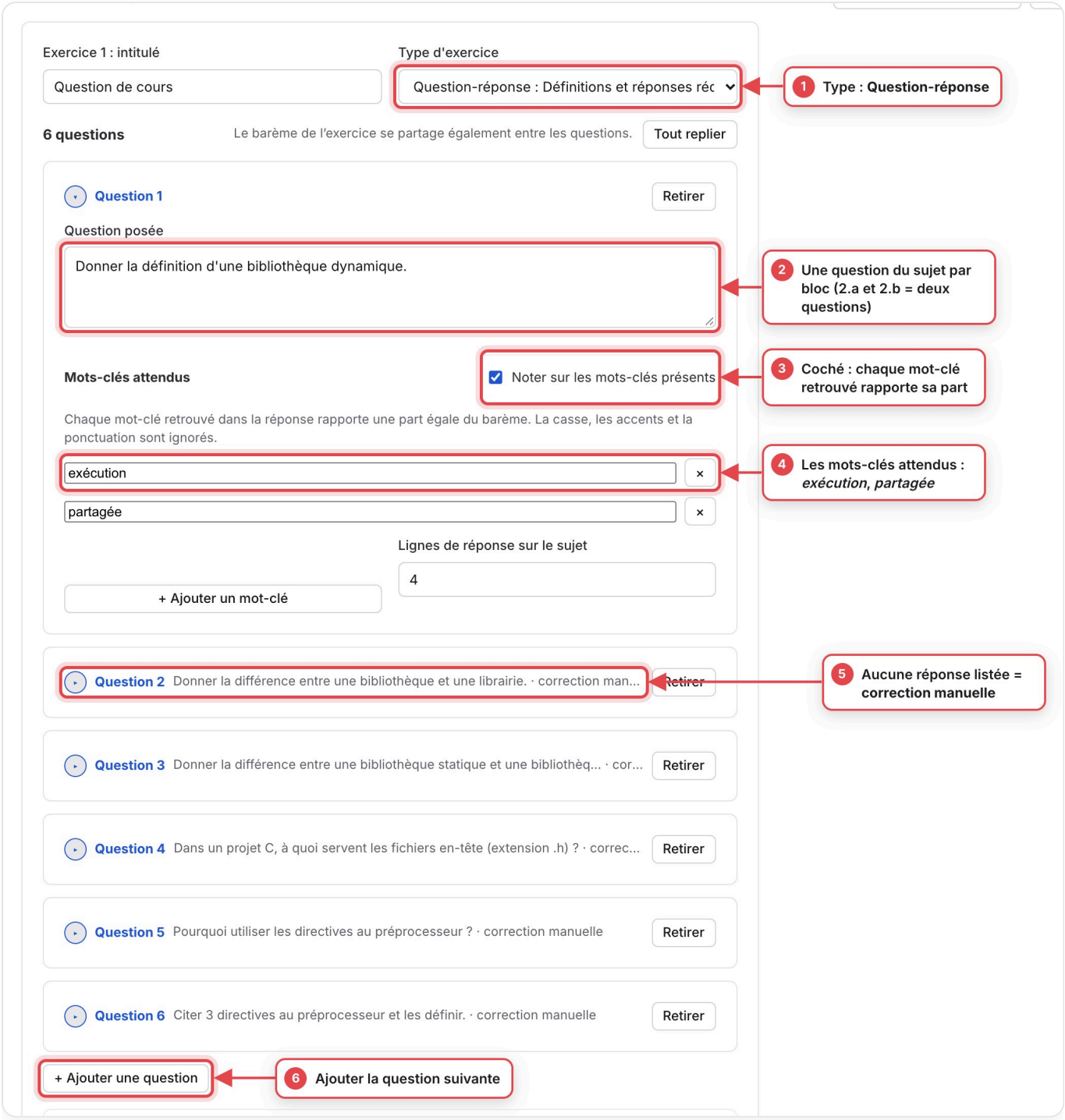
QUESTION-RÉPONSE Exercice 1 : question de cours (10 points)

SUR LE SUJET PAPIER

Exercice 1 Question de cours (10 points)

1. Donner la définition d'une bibliothèque dynamique.
2. Donner la différence entre :
   a. Une bibliothèque et une librairie
   b. Une bibliothèque statistique et une bibliothèque dynamique
3. Dans un projet C à quoi servent les fichiers entête (extension .h)
4. Pourquoi utiliser les directives au préprocesseur
5. Citer 3 directives au préprocesseur et les définir

Dans CodEval



1. Type **Question-réponse**.
2. Une question du sujet par bloc. Le 2.a et le 2.b deviennent deux questions : 6 questions au total, qui se partagent les 10 points.
3. **Noter sur les mots-clés présents** : chaque mot-clé retrouvé dans la réponse rapporte sa part.
4. Les mots-clés de la question 1 : *exécution*, *partagée*. Majuscules, accents et ponctuation sont ignorés.
5. Sans réponse listée, la question vous est signalée pour une **correction manuelle**.
6. **+ Ajouter une question** pour la suivante.

QUESTION-RÉPONSE Exercice 2 : gestion d'un stock (10 points)

SUR LE SUJET PAPIER

Exercice 2 programmation C (10 points)

Proposer une approche pour réaliser une application de gestion d'un stock. Votre approche devra être modulaire.

Bon courage !

Même type que l'exercice 1 : il n'y a rien de nouveau à apprendre.

1. Une seule question, qui reprend la consigne du sujet.
2. **Lignes de réponse sur le sujet** : 12, pour laisser de la place à une réponse longue.
3. **Aucune réponse listée** : la correction est manuelle.
4. Mettez la démarche attendue dans **Corrigé proposé aux étudiants · Explications** : modules produit et stock, fichiers .h et .c, gardes d'inclusion, compilation séparée.

> **Variante : une réponse en code**
> Si vous préférez que l'étudiant écrive le code de son approche, utilisez le type **Code** en plusieurs fichiers (produit.h, stock.h, stock.c, main.c), présenté dans la partie C.

C. Examen 2e session de C avancé

Mars 2025 · M1 MBDS, BIHAR, SIGL · 3 exercices · 20 points

Paramètres : type **Examen**, classe *M1 MBDS*, matière *C avancé*, 2h, 20 points.

CODE Exercices 1 et 2.1 : le programme modulaire (12 points)

SUR LE SUJET PAPIER

On se propose d'écrire un programme en C permettant de calculer la moyenne des notes d'une classe de 40 étudiants et d'afficher le nom et la note des 40 étudiants ainsi que la moyenne de la classe.

- Les structures des données se présentent comme suites :
  - Etudiant (matricule, nom, note) ;
  - Note (date, note)
- Les fonctions et procédures utilisées sont :
  - Procédure : AjouterNote (ValeurNote, &UnEtudiant)
  - Fonction : MoyenneNotes(&TabEtudiants) retourne un réel
  - Procédure : AfficherTout (&TabEtudiants)

Pour une organisation modulaire du code, on se propose de créer les 5 fichiers suivants :

Etudiant.h ; Note.h ; Fonctions.h ; Fonctions.c ; main.c

NB : (Quelques bout de code pour vous aider si et seulement si vous en avez besoin (pas obligatoires)).

- *(type*)malloc(count*sizeof(type)) ;*
- *#include <stdio.h> ;*
- *#include <stdlib.h> ;*
- *#ifndef __ ;*
- *#define __ ;*
- *#endif ;*

EXERCICE 1 ***(08 points)***

1) Donne le contenu des fichiers Etudiant.h ; Note.h, Fonctions.h, *(02 pts par fonction)*
2) Donne le contenu du fichier Fonctions.c *(02 pts)*

EXERCICE 2 ***(07 points)***

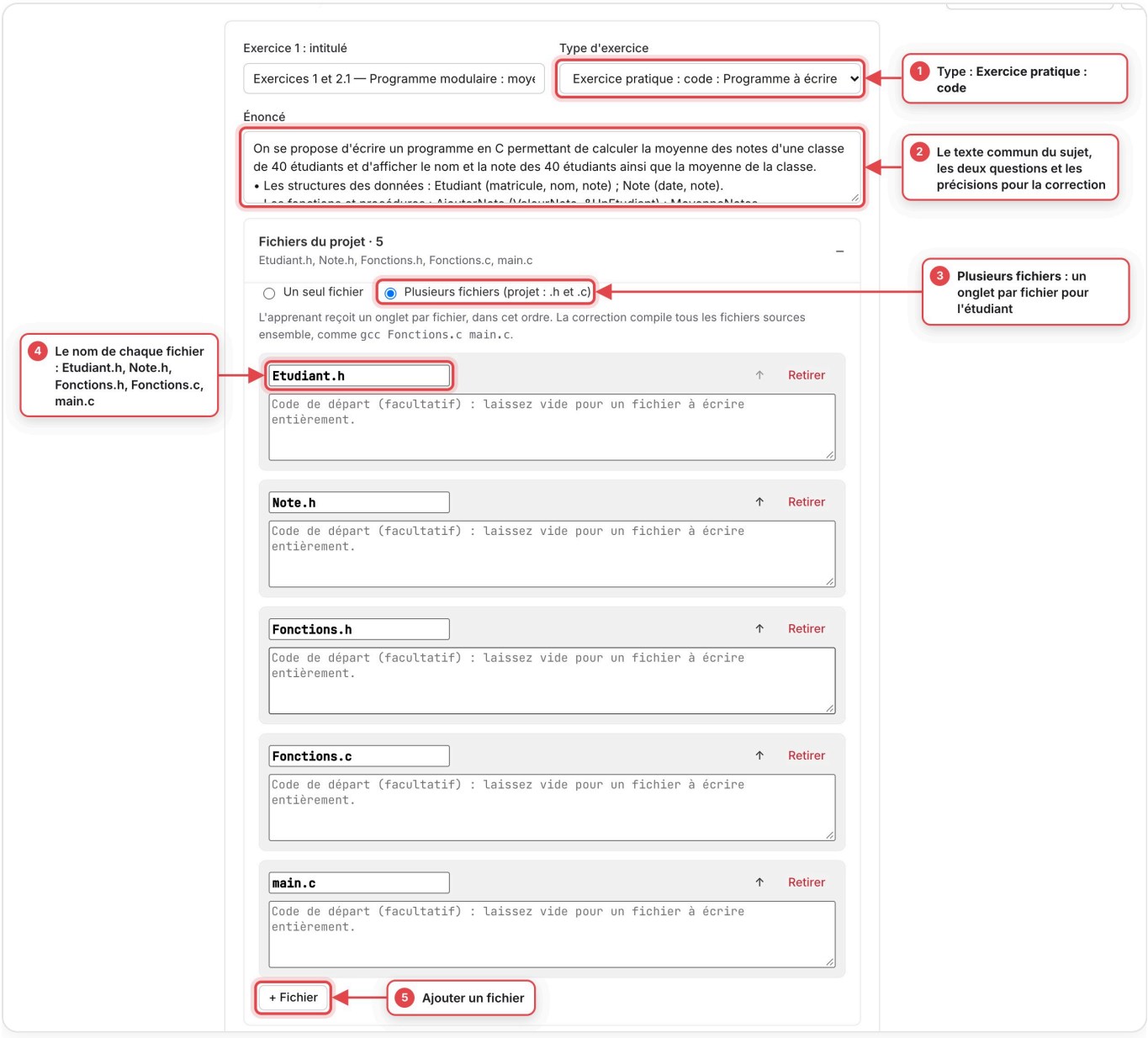
Soit la commande suivante permettant de lancer le programme final obtenu après les différentes phases de compilations :

c :> ProgrammeMoyenne.exe 40

1) Donne le contenu du fichier main.c *(04 pts)*
2) Donne la suite des commandes ayant permis d'obtenir le fichier ProgrammeMoyenne.exe *(03 pts)*

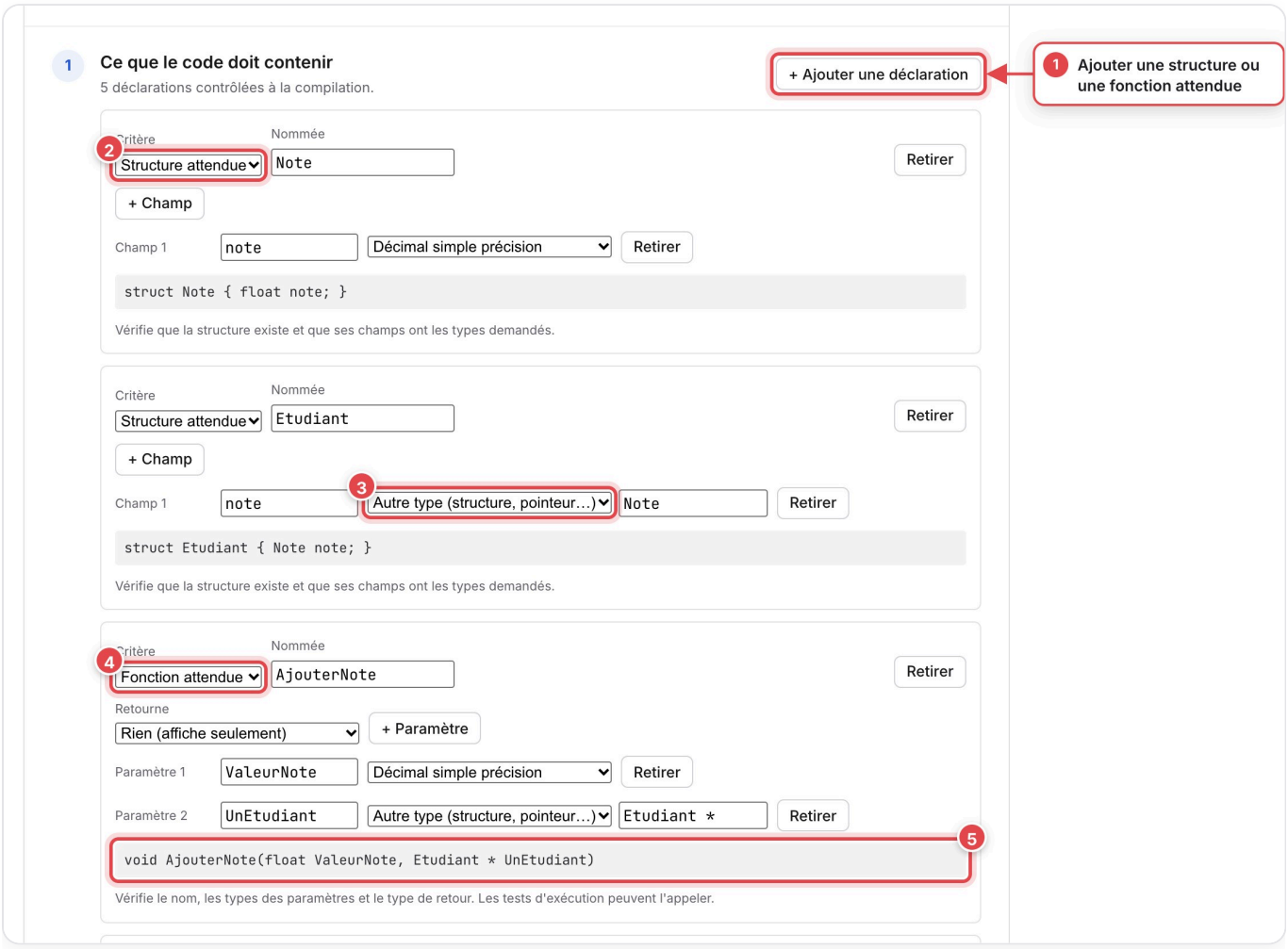
> **Pourquoi un seul exercice pour 1 et 2.1 ?**
>
> Les en-têtes (exercice 1) et main.c (exercice 2.1) forment **le même programme**. Réunis dans un seul exercice de 12 points (8 + 4), ils se compilent ensemble, comme `gcc Fonctions.c main.c`. L'exercice 2.2 (commandes) et le QCM restent des exercices à part.

Dans CodEval : l'énoncé et les fichiers du projet



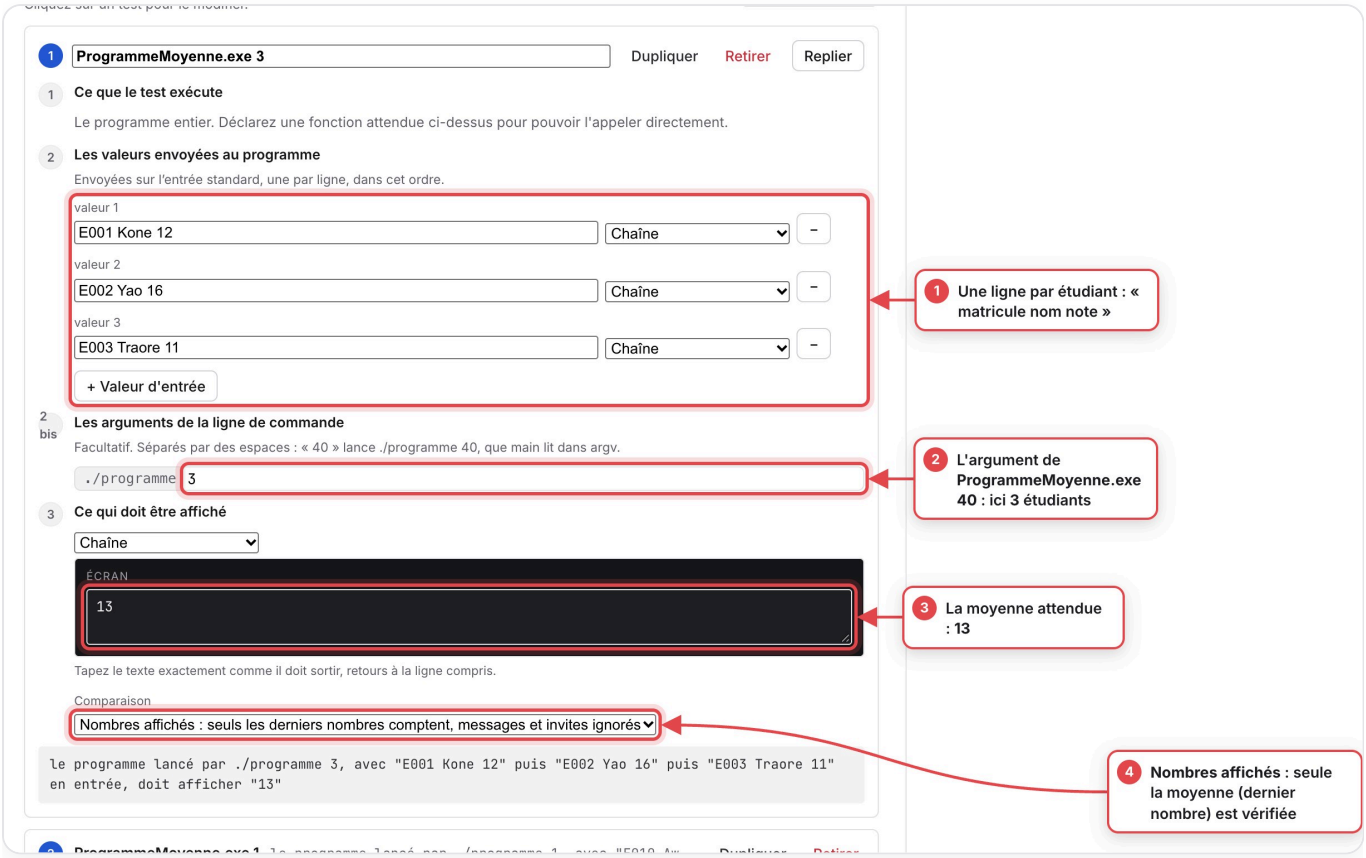
1. Type **Exercice pratique : code** (le langage C vient de la matière).
2. L'énoncé : le texte commun du sujet, les deux questions, et les précisions dont la correction automatique a besoin. Ce sont les signatures exactes, le nombre d'étudiants lu dans `argv[1]`, et une ligne « matricule nom note » par étudiant.
3. Ouvrez **Fichiers du projet** et choisissez **Plusieurs fichiers**.
4. Le nom de chaque fichier : Etudiant.h, Note.h, Fonctions.h, Fonctions.c, main.c. L'étudiant aura un onglet par fichier.
5. **+ Fichier** pour en ajouter un.

Étape « Correction » : ce que le code doit contenir



1. **+ Ajouter une déclaration** : une par structure ou fonction du sujet.
2. **Structure attendue** : *Note* (champ note : décimal), puis *Etudiant*.
3. Pour un type défini dans le projet, choisissez **Autre type** et écrivez-le comme en C : `Note`, `Etudiant *`.
4. **Fonction attendue** : *AjouterNote*, *MoyenneNotes*, *AfficherTout*, avec le type de retour et les paramètres.
5. La signature vérifiée s'affiche en clair. La correction la cherche dans **tous les fichiers** du projet.

Étape « Correction » : le programme lancé avec un argument

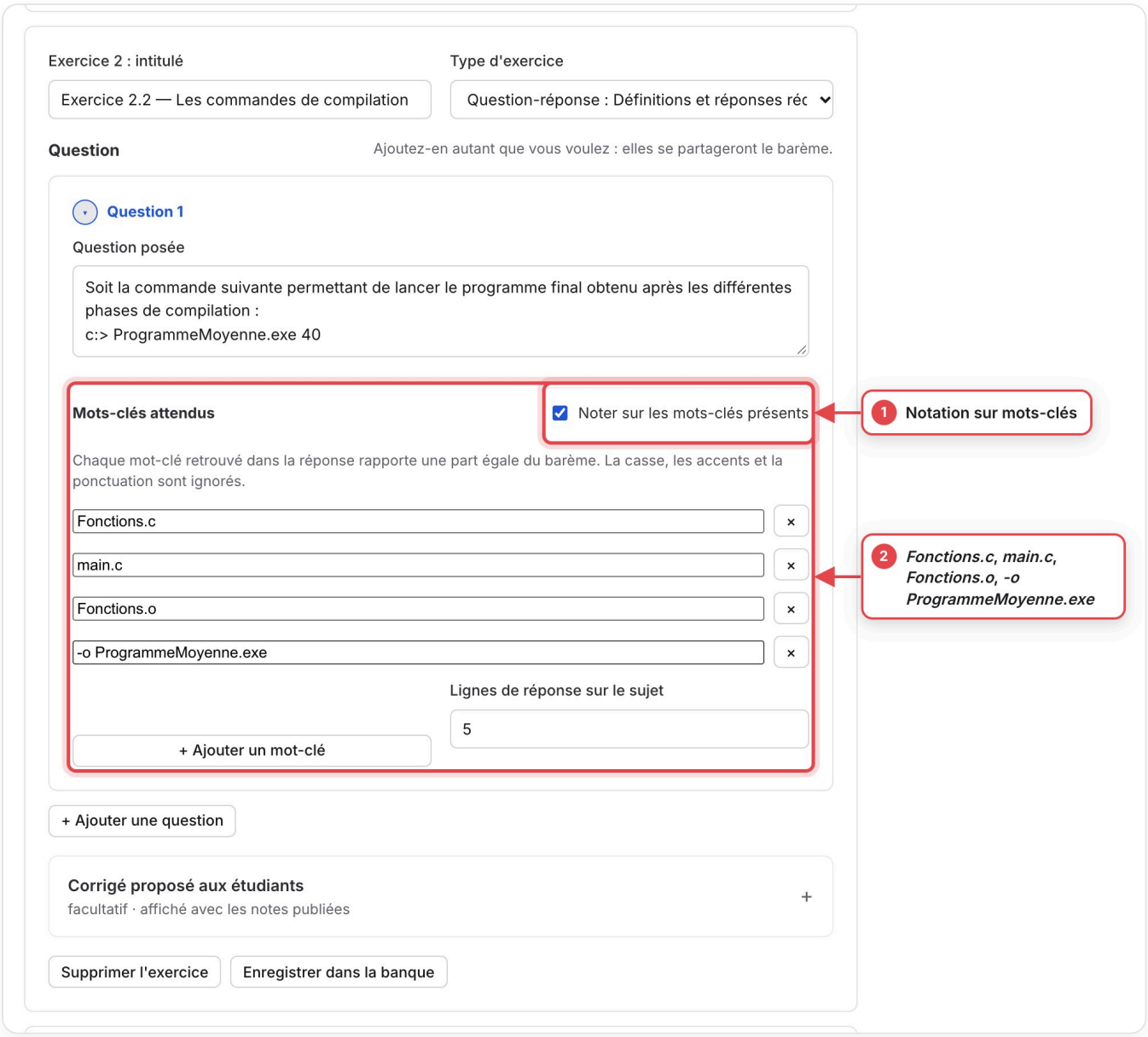


1. Les valeurs saisies : une ligne par étudiant, « matricule nom note ».
2. L'**argument de la ligne de commande**, qui remplace le `40` de `ProgrammeMoyenne.exe 40` : ici `3` étudiants.
3. L'attendu : la moyenne de la classe, `13`.
4. **Nombres affichés** : seule la moyenne (le dernier nombre affiché) est vérifiée. La présentation de la liste reste libre.

> **Contrôlé avec la solution de référence**
>
> Les 5 déclarations et les 2 tests (lancés avec `3` puis `1`) donnent 12/12. À l'étape 4, **Détail du barème** répartit les 12 points : 6 pour les déclarations (en-têtes), 6 pour les tests (Fonctions.c et main.c).

QUESTION-RÉPONSE Exercice 2.2 : les commandes de compilation (3 points)

Même type que les questions de cours, avec la notation sur mots-clés :



1. **Noter sur les mots-clés présents** coché.
2. Mots-clés : `Fonctions.c`, `main.c`, `Fonctions.o`, `-o ProgrammeMoyenne.exe`. Une compilation séparée suivie de l'édition de liens obtient 3/3. Une seule commande (`gcc Fonctions.c main.c -o ProgrammeMoyenne.exe`) obtient 2,25/3 : il manque le fichier objet.

QCM Exercice 3 : QCM (5 points)

SUR LE SUJET PAPIER

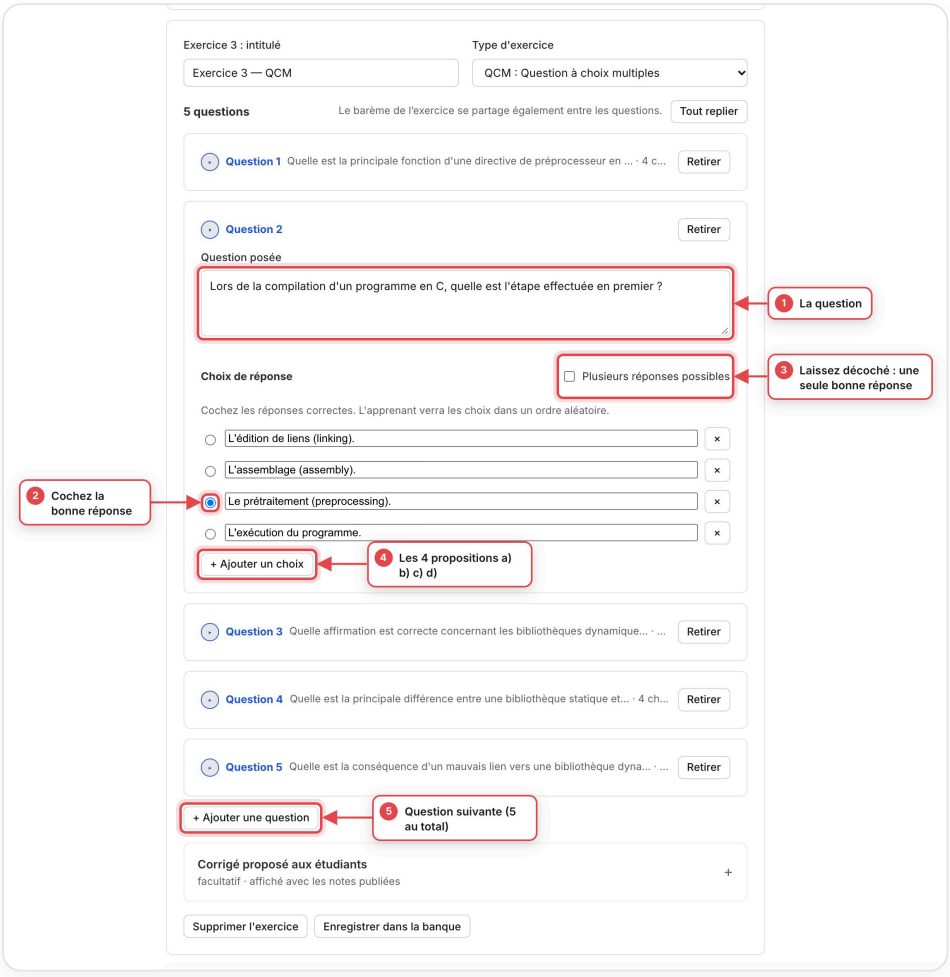
EXERCICE 3 *(05 points)*

1. Quelle est la principale fonction d'une directive de préprocesseur en C ?

a) Modifier le code source avant la compilation.
b) Améliorer les performances du programme.
c) Ajouter de nouvelles fonctionnalités au langage C.
d) Remplacer automatiquement les erreurs de syntaxe.

2. Lors de la compilation d'un programme en C, quelle est l'étape effectuée en premier ?

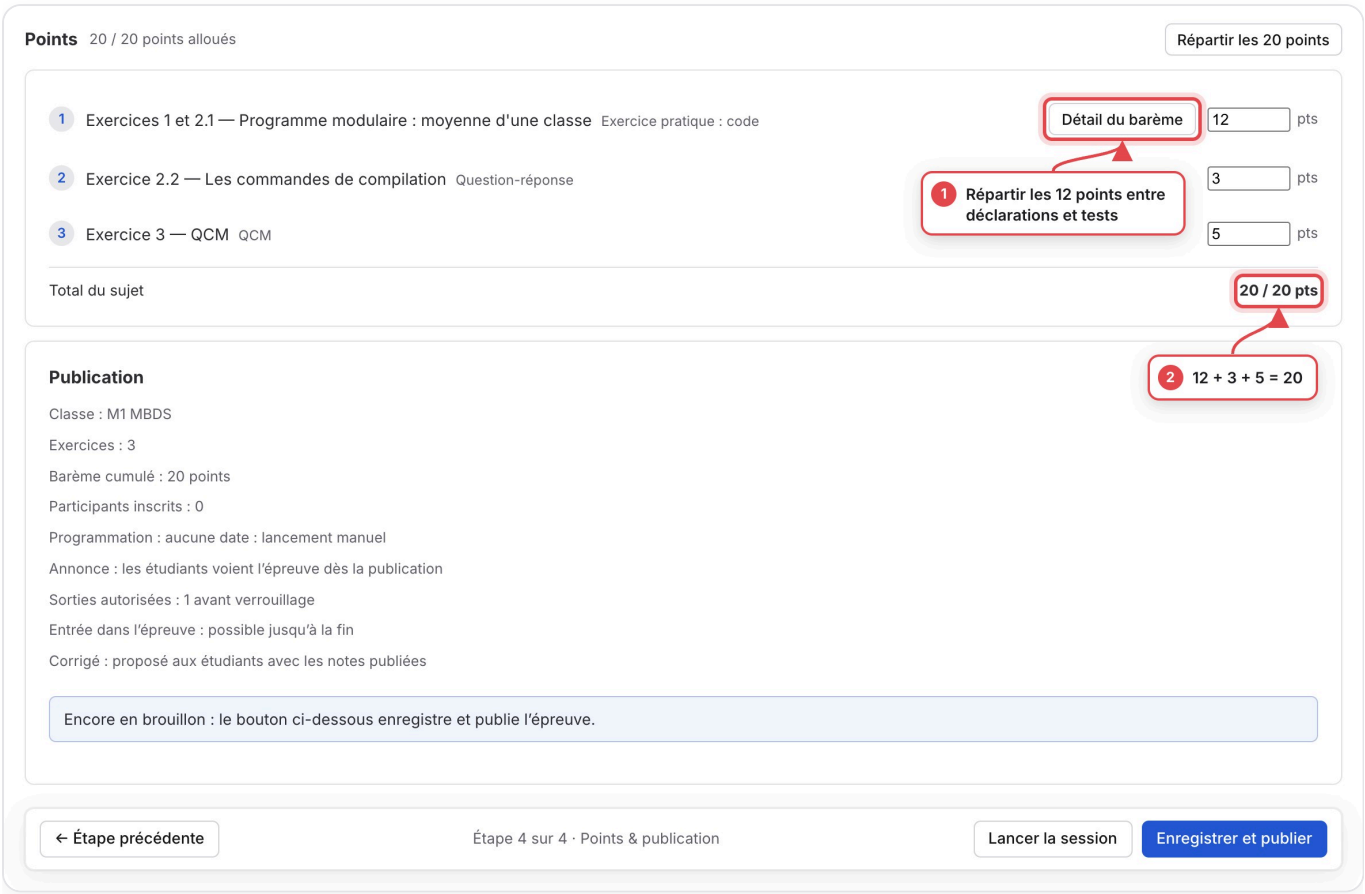
a) L'édition de liens (linking).
b) L'assemblage (assembly).
c) Le prétraitement (preprocessing).
d) L'exécution du programme.



1. La question.
2. Cochez la **bonne réponse** (bouton rond).
3. Laissez **Plusieurs réponses possibles** décoché : une seule bonne réponse.
4. Les quatre propositions a) b) c) d). L'étudiant les voit dans un ordre mélangé.
5. **+ Ajouter une question** : 5 questions, qui se partagent les 5 points.

> **Question 5 du PDF**
> Les propositions b) et c) y sont collées sur une même ligne. Saisissez-les comme deux choix séparés.

Étape 4 « Points & publication »



1. **Détail du barème** : répartir les 12 points du projet entre déclarations et tests.
2. **12 + 3 + 5 = 20**. Puis **Enregistrer et publier**.

D. Après l'épreuve

De la session à la copie corrigée.

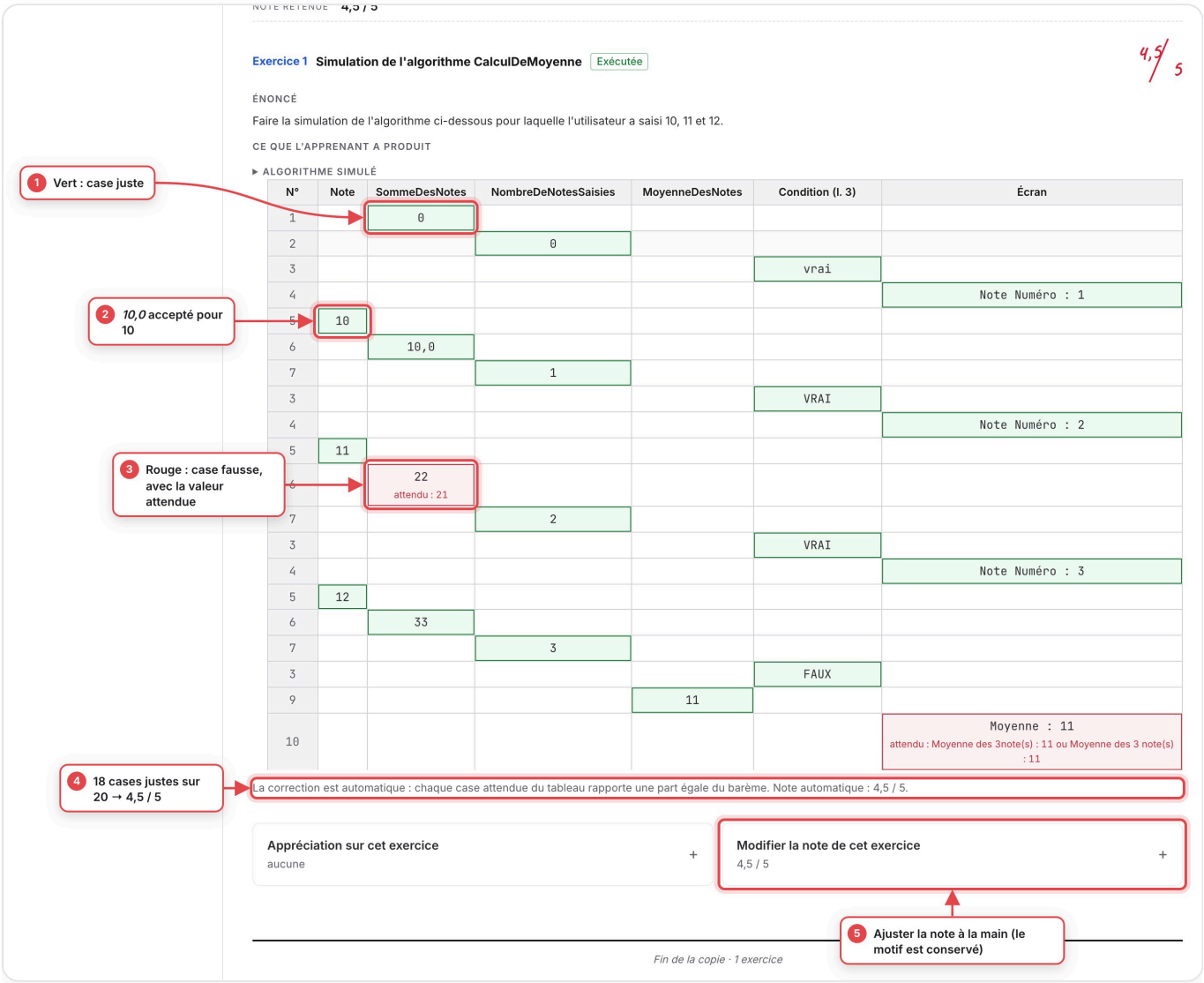
1 — Session
Lancer la session, suivre les présents, prolonger si besoin

2 — Clôture
À la fin du temps ou à la main : les copies sont figées

3 — Correction
Lancer la correction, relire chaque copie, ajuster

4 — Publication
Publier les résultats : chaque étudiant voit sa copie et le corrigé

Une copie de simulation corrigée



1. **Vert** : la case est juste.
2. Une valeur équivalente est acceptée : `10,0` pour 10, `vrai` pour VRAI.
3. **Rouge** : la case est fausse, et la valeur attendue est indiquée dessous.
4. La note calculée : 18 cases justes sur 20, soit **4,5 / 5**.
5. **Modifier la note de cet exercice** : ajustez à la main, par exemple pour une question en correction manuelle ou pour le point de propreté. Le motif est demandé et conservé.

E. Aide-mémoire

Type	Ce que vous saisissez	Comment c'est corrigé
Simulation	L'algorithme, puis le tableau attendu (colonnes, lignes, colonnes fournies)	Case par case ; case vide non notée ; nombres par valeur ; `|` pour plusieurs réponses
Algorithmique	Énoncé, blocs autorisés, puis des tests (valeurs envoyées, nombres attendus)	Exécution sur chaque test ; *Nombres affichés* recommandé
Code (plusieurs fichiers)	Énoncé, noms des fichiers, déclarations attendues, tests avec arguments	Signatures cherchées dans tout le projet, puis programme compilé et exécuté
Question-réponse	Une question par bloc ; mots-clés, formulations acceptées, ou rien	Mots-clés ou formulations ; sinon correction manuelle
QCM	Questions, propositions, bonne(s) réponse(s)	Automatique ; propositions mélangées pour l'étudiant

Avant de publier

☐	La classe et la matière sont choisies (sans classe, l'épreuve reste un brouillon invisible).
☐	Chaque exercice pratique a au moins un test, marqué *Prêt* à l'étape Correction.
☐	L'ordre de saisie des données est écrit dans l'énoncé quand il y a plusieurs valeurs à lire.
☐	Le tableau de simulation a une ligne par instruction exécutée, et les bonnes colonnes fournies.
☐	Le total des points fait 20 (étape 4).
☐	L'aperçu du sujet, à droite de l'étape « Exercices », se lit comme le sujet papier.

> **Réutiliser un exercice**
> Le bouton *Enregistrer dans la banque*, sur la carte d'un exercice, le range dans votre **Banque d'exercices**. Vous le retrouverez avec *Importer depuis la banque* dans une prochaine épreuve.